

Fluxbase

Developer Integration & Architecture Manual

Complete Reference for REST SQL, AI Gateway, MCP, WebSockets & Storage

v4.2 Production

PostgreSQL

MySQL

Flux AI Gateway

Model Context Protocol (MCP)

REST + WS

Production Gateway Specifications

Core SQL Endpoint:	POST https://www.fluxbasedb.me/api/execute-sql
AI Gateway (Chat):	POST https://www.fluxbasedb.me/api/v1/chat/completions
MCP JSON-RPC:	POST https://www.fluxbasedb.me/api/mcp
Real-Time WebSockets:	wss://fluxbase-realtime.onrender.com
Authentication:	Authorization: Bearer <FLUXBASE_API_KEY>
Default AI Model:	flux (Flagship reasoning model family)

1. Getting Started & Architecture

Fluxbase is a unified backend database and AI infrastructure platform providing instant serverless databases, an OpenAI-compatible AI gateway, Model Context Protocol (MCP) tooling for autonomous coding agents, sub-second real-time streaming, AWS S3-backed file storage, and enterprise Row-Level Security (RLS).

Required Integration Credentials

Every Fluxbase integration requires three project parameters from your Project Settings:

- 1. Project ID: Unique 16-character identifier (e.g., 51c04beb753a42f3)
- 2. API Key: Scoped Bearer token (flx_live_...) with read, write, ai, or admin permissions
- 3. Base URL: https://www.fluxbasedb.me (for all REST & Gateway APIs)

Environment Configuration

Store your secrets in server-only environment variables. Never expose your live API key to the client browser.

BASH

```
# .env.local (Server Only)
FLUXBASE_API_KEY=flx_live_XXXXXXXXXXXXXXXXXXXX
FLUXBASE_PROJECT_ID=51c04beb753a42f3
FLUXBASE_BASE_URL=https://www.fluxbasedb.me
NEXT_PUBLIC_WS_URL=wss://fluxbase-realtime.onrender.com
```

Authentication & Scope Hierarchy

All requests must include an Authorization header with Bearer format. Requests without valid keys return 401 Unauthorized.

HTTP

```
Authorization: Bearer flx_live_XXXXXXXXXXXXXXXXXXXX
Content-Type: application/json
```

Key Scopes

Scope	Allowed Operations	Recommended Environment
read	SELECT queries only	Public dashboards, read-only analytics
write	SELECT, INSERT, UPDATE, DELETE	Backend services, user mutations
ai	Flux AI Gateway chat completions	Cursor, AI Agents, LLM services
admin	Full DDL (CREATE, ALTER, DROP) + DML	Migrations, seed scripts, CLI setup

2. Flux AI Gateway (OpenAI-Compatible)

Fluxbase provides a high-throughput, drop-in OpenAI-compatible AI gateway. Connect standard OpenAI SDKs, LangChain, Cursor, Windsurf, or custom agent frameworks to Flux AI reasoning models without changing code.

Gateway Base URL & Compatibility

Endpoint: POST <https://www.fluxbasedb.me/api/v1/chat/completions>
Set your client base_url to "https://www.fluxbasedb.me/api/v1" and supply your Fluxbase API Key.
Dedicated Interactive Hub: <https://www.fluxbasedb.me/ai-models>
Supports streaming (stream=true), image generation, video tasks, and speech STT/TTS.

Frontier Model Catalog & Real Upstream Architectures

Gateway ID	Real Upstream Model	Provider / Tier	Specs & Best Use Case
flux-sonnet	Claude Sonnet 4.5	Bedrock (Pro)	Frontier reasoning, complex coding, 200k context, 64k max output
flux-pro-max	Claude 3.7 / 4.6 Sonnet	Bedrock (Max)	Extended thinking benchmarks and deep SQL query optimization
flux-turbo	LLaMA 3.3 70B Versatile	Groq (Free)	Hyper-speed 300+ tok/s inference on Groq LPUs. 128k context
flux-omni	Gemini 2.0 Flash	Google (Free)	Massive 1M token context window, multimodal vision understanding
flux-max	GPT-4o Mini	OpenAI (Free)	Lightweight flagship coding and prompt adherence. 128k context
flux	GLM-4 Flash (Default)	Zhipu (Free)	High-accuracy general reasoning, precision SQL synthesis. 128k
flux-flash	GLM-4 Flash (Turbo UX)	Zhipu (Free)	Sub-100ms token throughput. Ideal for autocompletion and chat UX
flux-5.2	GLM-4 Plus (Agentic)	Zhipu (Free)	Specialized reasoning for autonomous multi-step agent tool
flux-image-ultra	Stable Image Ultra 1.0	Bedrock (Pro)	SOTA photorealism, typography rendering, S3 auto cloud storage
flux-image-hd	Imagen 3.0	Google (Free)	High-definition 4K image generation with superior detail rendering
flux-video-ray	Luma Ray v2	Bedrock (Pro)	Cinema-grade video generation with realistic physics and motion
flux-listen	Whisper Large v3 Turbo	Groq (Free)	10x real-time multilingual transcription with word-level timestamps
flux-speak	TTS-1 (6 Voices)	OpenAI (Free)	Natural expressive speech synthesis (alloy, echo, fable, onyx, etc.)
flux-embed	Text Embedding 004	Google (Free)	768-dimensional dense vector embeddings for semantic search &
gpt-4o	GPT-4o Alias	OpenAI (Free)	Drop-in compatibility alias automatically mapped to flux-ultra

Tier Token Allocations & Rate Limits Matrix

Plan Tier	Text TPM	Requests / Min	Daily Quota	Image & Video Limits
Free	10,000 TPM	10 RPM	500 req / day	5 Images/day • Video locked
Pro	100,000 TPM	60 RPM	10,000 req / day	50 Images/day • 5 Videos/day
Max	500,000 TPM	300 RPM	50,000 req / day	200 Images/day • 20 Videos/day
Pay-As-You-Go	Unlimited TPM	Unlimited RPM	Unlimited (Metered)	Uncapped elasticity at standard rates

Python Integration (OpenAI SDK)

PYTHON

```

from openai import OpenAI

client = OpenAI(
    base_url="https://www.fluxbasedb.me/api/v1",
    api_key="flx_live_your_fluxbase_key"
)

# 1. Frontier Reasoning with Claude Sonnet 4.5
response = client.chat.completions.create(
    model="flux-sonnet",
    messages=[
        {"role": "system", "content": "You are a senior database architect."},
        {"role": "user", "content": "Design an optimal distributed sharding schema."}
    ],
    temperature=0.2
)
print(response.choices[0].message.content)

# 2. SOTA Photorealistic Image Generation
img = client.images.generate(
    model="flux-image-ultra",
    prompt="Futuristic data center server room, neon orange coolant, cinematic 8k",
    size="1024x1024"
)
print("Image URL:", img.data[0].url)

```

Node.js / TypeScript Integration

TYPESCRIPT

```

import OpenAI from 'openai';

const client = new OpenAI({
  baseURL: 'https://www.fluxbasedb.me/api/v1',
  apiKey: process.env.FLUXBASE_API_KEY,
});

async function generateSchema() {
  const stream = await client.chat.completions.create({
    model: 'flux',
    messages: [{ role: 'user', content: 'Generate a PostgreSQL partitioned table schema.' }],
    stream: true,
  });
  for await (const chunk of stream) {
    process.stdout.write(chunk.choices[0]?.delta?.content || '');
  }
}

```

3. Model Context Protocol (MCP) Gateway

Fluxbase implements the Model Context Protocol (MCP) over JSON-RPC 2.0. This allows AI assistants such as Google Antigravity, Cursor, Windsurf, and Claude Desktop to introspect live schemas, run migrations, and execute queries autonomously with zero manual prompting.

MCP Gateway Endpoint

POST <https://www.fluxbasedb.me/api/mcp>
Protocols: JSON-RPC 2.0 over HTTP POST. Supports tools/list and tools/call methods.
Authenticated via Authorization: Bearer <API_KEY>

Available MCP Tools

Tool Name	Category	Description
create_project	PROJECT	Provisions new serverless PostgreSQL or MySQL database projects
list_projects	DISCOVERY	Lists all database projects owned by or shared with your account
get_schema	SCHEMA	Inspects live table structures, column definitions, data types, and primary keys
run_sql	EXECUTE	Executes raw SQL DDL and DML queries against the active project database

Connecting with Google Antigravity

Antigravity supports MCP natively via mcp_config.json. Add this to .agents/mcp_config.json in your workspace root:

JSON

```
{
  "mcpServers": {
    "fluxbase": {
      "serverUrl": "https://www.fluxbasedb.me/api/mcp",
      "headers": {
        "Authorization": "Bearer flx_live_your_fluxbase_key"
      }
    }
  }
}
```

Connecting with Cursor, Windsurf & Claude Desktop

In Cursor or Windsurf (Settings ! Features ! MCP ! Add Server) or Claude Desktop

JSON

```
{
  "mcpServers": {
    "fluxbase": {
      "url": "https://www.fluxbasedb.me/api/mcp",
      "headers": {
        "Authorization": "Bearer flx_live_your_fluxbase_key"
      }
    }
  }
}
```

CLI Discovery Test (Node.js)

BASH

```
node -e "fetch('https://www.fluxbasedb.me/api/mcp', {
  method: 'POST',
  headers: { 'Authorization': 'Bearer YOUR_KEY', 'Content-Type': 'application/json' },
  body: JSON.stringify({ jsonrpc: '2.0', id: 1, method: 'tools/list' })
}).then(r => r.json()).then(d => console.log(JSON.stringify(d, null, 2)))"
```

4. Core SQL API & Execution Engine

Execute any SQL statement against your project database via a single unified endpoint. The Fluxbase engine supports full PostgreSQL syntax, Common Table Expressions (CTEs), window functions, timestamp arithmetic, parameterized queries, and automatic transaction handling.

Endpoint Specification

HTTP

```
Method:  POST
URL:     https://www.fluxbasedb.me/api/execute-sql
Header:  Authorization: Bearer <API_KEY>
Header:  Content-Type: application/json
```

Request & Response Format

JSON

```
// Request Body
{
  "projectId": "51c04beb753a42f3",
  "query": "SELECT id, name, email FROM users WHERE active = $1 LIMIT 5;",
  "params": [true]
}

// Success Response (HTTP 200)
{
  "success": true,
  "result": {
    "rows": [{ "id": 1, "name": "Alice", "email": "alice@example.com" }],
    "columns": ["id", "name", "email"],
    "rowCount": 1,
    "message": null
  },
  "executionInfo": { "time": "9ms", "rowCount": 1, "operation": "SELECT" }
}
```

Advanced SQL: Window Functions & CTEs

Fluxbase fully executes complex analytical queries with LAG, LEAD, and epoch calculations:

SQL

```
WITH ordered_predictions AS (  
  SELECT *,  
         LAG(timestamp) OVER (ORDER BY timestamp) AS prev_timestamp  
  FROM predictions  
)  
,  
time_differences AS (  
  SELECT timestamp,  
         prev_timestamp,  
         EXTRACT(EPOCH FROM (timestamp - prev_timestamp)) AS time_diff_seconds  
  FROM ordered_predictions  
  WHERE prev_timestamp IS NOT NULL  
)  
SELECT MAX(time_diff_seconds) AS longest_shutdown_time_seconds  
FROM time_differences;
```

Execution Rule

SQL runtime errors return HTTP 200 with { success: false, error: { message, code: "SQL_EXEC_ERROR" } }.

Always verify the success boolean in application code rather than relying only on HTTP status.

5. Language SDKs & Code Snippets

Fluxbase is a standard RESTful service. Use native HTTP clients in any programming language.

Node.js / TypeScript (Native fetch)

JAVASCRIPT

```
const BASE_URL = 'https://www.fluxbasedb.me';
const API_KEY = process.env.FLUXBASE_API_KEY;
const PROJECT = process.env.FLUXBASE_PROJECT_ID;

async function query(sql, params = []) {
  const res = await fetch(`${BASE_URL}/api/execute-sql`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${API_KEY}` },
    body: JSON.stringify({ projectId: PROJECT, query: sql, params })
  });
  const json = await res.json();
  if (!json.success) throw new Error(json.error?.message ?? 'Query failed');
  return json.result.rows;
}

const users = await query('SELECT * FROM users WHERE active = $1', [true]);
```

Python (requests)

PYTHON

```
import os, requests

URL = 'https://www.fluxbasedb.me/api/execute-sql'
HEADERS = { 'Authorization': f'Bearer {os.getenv("FLUXBASE_API_KEY")}', 'Content-Type':
'application/json' }

def query(sql, params=None):
    payload = {'projectId': os.getenv('FLUXBASE_PROJECT_ID'), 'query': sql}
    if params: payload['params'] = params
    resp = requests.post(URL, json=payload, headers=HEADERS).json()
    if not resp.get('success'): raise Exception(resp.get('error', {}).get('message'))
    return resp['result']['rows']

rows = query('SELECT * FROM users LIMIT 10')
```

Go (net/http)

GO

```
body, _ := json.Marshal(map[string]any{
    "projectId": os.Getenv("FLUXBASE_PROJECT_ID"),
    "query":     "SELECT * FROM users LIMIT 10",
})
req, _ := http.NewRequest("POST", "https://www.fluxbasedb.me/api/execute-sql", bytes.NewBuffer(body))
req.Header.Set("Authorization", "Bearer " + os.Getenv("FLUXBASE_API_KEY"))
req.Header.Set("Content-Type", "application/json")
resp, err := http.DefaultClient.Do(req)
```

cURL

BASH

```
curl -X POST "https://www.fluxbasedb.me/api/execute-sql" \
  -H "Authorization: Bearer $FLUXBASE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"projectId":"YOUR_PROJECT_ID","query":"SELECT * FROM users LIMIT 5;"}'
```

6. Real-Time WebSockets & Streaming

Fluxbase broadcasts database change events in real-time. Whenever an INSERT, UPDATE, DELETE, or DDL migration occurs, change events stream directly to connected clients within 10–25 milliseconds.

WebSocket Gateway Architecture

Production WebSocket URL: wss://fluxbase-realtime.onrender.com

Room Convention: "project_<PROJECT_ID>"

Clients subscribe to project rooms and automatically receive filtered table mutation events.

Browser / Node.js WebSocket Example

JAVASCRIPT

```
const ws = new WebSocket('wss://fluxbase-realtime.onrender.com');

ws.onopen = () => {
  // Subscribe to project room
  ws.send(JSON.stringify({
    type: 'subscribe',
    roomId: 'project_YOUR_PROJECT_ID'
  }));
};

ws.onmessage = (event) => {
  const msg = JSON.parse(event.data);
  if (msg.type === 'db_event') {
    console.log(`[${msg.payload.operation}] on table ${msg.payload.table}:`, msg.payload.record);
  }
};
```

Real-time Event Schema

Message Type	Payload Operation	Trigger Event
subscribed	—	Server acknowledges subscription to project channel
db_event	INSERT	A new row was added via SQL API or Table Editor
db_event	UPDATE	An existing row was modified
db_event	DELETE	A row was removed from a table
db_event	schema_update	DDL executed: CREATE, ALTER, DROP, or TRUNCATE TABLE

7. Storage v2 (AWS S3-Backed File Engine)

Fluxbase Storage provides isolated S3 buckets, private-by-default access, and pre-signed temporary URLs. Store user avatars, PDF invoices, datasets, media files, and backups securely.

File Operations

Method	Endpoint	Description
POST	/api/storage/buckets	Create a bucket { projectId, name, isPublic }
GET	/api/storage/buckets?projectId=...	List all buckets and size rollups
POST	/api/storage/upload	Upload file via multipart/form-data (file, bucketId, projectId)
GET	/api/storage/url?s3Key=...&projectId=...	Generate 15-minute secure pre-signed download URL
DELETE	/api/storage/files	Delete file from S3 and metadata database

Uploading via JavaScript

JAVASCRIPT

```
const formData = new FormData();
formData.append('file', fileInput.files[0]);
formData.append('bucketId', 'avatars');
formData.append('projectId', 'YOUR_PROJECT_ID');

const res = await fetch('https://www.fluxbasedb.me/api/storage/upload', {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${API_KEY}` },
  body: formData
});
const { file } = await res.json();
// file.s3_key is saved to your table for later URL retrieval
```

Supported MIME Types & Tier Limits

- › Images: JPEG, PNG, GIF, WebP, SVG, AVIF
- › Documents: PDF, TXT, CSV, JSON, Markdown
- › Media: MP4, WebM, MP3, WAV, ZIP archives
- › Tier Limits: Free (50 MB/file), Pro (500 MB/file), Enterprise (2 GB/file)

8. Team Collaboration & Webhooks

Manage project collaborators, role-based access, and outbound webhooks programmatically.

Team Management API

HTTP

```
GET    /api/team?projectId=YOUR_PROJECT_ID           // List active members & pending invites
POST   /api/team                                   // Invite member: { projectId, email, role }
DELETE /api/team?projectId=...&userId=...           // Remove user from project
POST   /api/team/invites/accept                     // Accept invite: { inviteId, status:
"accepted" }
```

Role-Based Access Control (RBAC)

Role	Data Access	Administrative Privileges
admin	Full SELECT, INSERT, UPDATE, DELETE	Manage billing, API keys, team members, and settings
developer	Full SELECT, INSERT, UPDATE, DELETE	Manage schemas, execute queries, view logs. No billing
viewer	Read-only (SELECT)	Inspect dashboard, schema, and read data. No modifications

Outbound Webhooks

Fluxbase posts JSON events directly to your serverless or backend endpoint on table mutations.

JSON

```
POST /api/fluxbase-webhook
X-Fluxbase-Signature: sha256=d58e37... (HMAC-SHA256 signature)
Content-Type: application/json

{
  "event_type": "row.inserted",
  "project_id": "YOUR_PROJECT_ID",
  "table_id": "orders",
  "timestamp": "2026-09-18T20:00:00.000Z",
  "data": {
    "new": { "id": 101, "amount": 89.50, "status": "paid" },
    "old": null
  }
}
```

Signature Verification in Node.js

JAVASCRIPT

```
import crypto from 'crypto';

export async function POST(req) {
  const rawBody = await req.text();
  const sig = req.headers.get('x-fluxbase-signature') ?? '';
  const secret = process.env.FLUXBASE_WEBHOOK_SECRET;
  const expected = 'sha256=' + crypto.createHmac('sha256', secret).update(rawBody).digest('hex');
  if (sig !== expected) return new Response("Unauthorized", { status: 401 });
  // Process webhook payload safely...
  return new Response("OK", { status: 200 });
}
```

9. Error Codes & Row-Level Security

Standardized Error Codes

Status	Error Code	Remediation Guide
401	AUTH_REQUIRED	Missing Authorization: Bearer <key> header
401	TOKEN_EXPIRED	JWT session or API key has expired. Re-authenticate
403	SCOPE_MISMATCH	API key lacks required scope (e.g., ai key used for DDL)
403	PROJECT_SUSPENDED	Project is suspended due to quota limits or billing
404	PROJECT_NOT_FOUND	Verify project identifier in Settings ! Project ID
429	RATE_LIMIT	Exceeded 50 req/10s per project. Use exponential backoff
200	SQL_EXEC_ERROR	Database execution syntax error. Inspect error.details
500	INTERNAL_ERROR	Unexpected server condition. Auto-logged for monitoring

Row-Level Security (RLS) & Multi-Tenant Isolation

Fluxbase injects the authenticated caller ID via PostgreSQL session state before query execution: SET LOCAL fluxbase.auth_uid = \$1. This ensures data isolation at the database engine level.

Common RLS Policy Patterns

- > User Owns Their Data: USING (user_id = auth.uid())
- > Organization Multi-Tenancy: USING (org_id = auth.uid())
- > Public Read / Owner Write: Policy 1 SELECT (true), Policy 2 UPDATE (author_id = auth.uid())
- > Full Internal Lockdown: USING (false) — prevents all external API access to sensitive audit tables

Interactive Documentation & Support

Interactive Documentation: <https://www.fluxbasedb.me/docs>
Direct PDF Download: <https://www.fluxbasedb.me/api/docs/download-pdf>
Technical Support: sumithsumith4567890@gmail.com